

Constraint-Based Problem Solving

Comprehensive Report — Questions 1, 2 and 3

Computer Science and Engineering, SIMATS Engineering

Course: Artificial Intelligence (CSA17)

1. Introduction

This report documents the analysis, solution methodology, and program execution results for three constraint-based problem-solving exercises in Artificial Intelligence: a Constraint Satisfaction Problem (CSP) solved via Backtracking Search, a shortest-path problem solved via Breadth-First Search (BFS), and an online navigation problem solved via Uniform Cost Search (UCS) combined with an Online Search Agent framework. Each section presents the problem statement, the solution approach with a full step-by-step trace, and the corresponding program output.

2. Question 1 — Hospital Shift Scheduling (CSP)

2.1 Problem Statement

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts — Morning, Afternoon, and Night — such that all of the following constraints are satisfied simultaneously:

- (i) D1 cannot work the Night shift.
- (ii) D2 must work before D3, where the shift ordering is Morning < Afternoon < Night.
- (iii) Only one doctor may be assigned per shift (all assignments must be distinct).
- (iv) D3 cannot work the Morning shift.
- (v) Every doctor must be assigned exactly one shift (no doctor is left unassigned).

Task: Apply Backtracking Search to find a valid assignment, showing all intermediate steps and constraint checks.

2.2 Solution Approach — Backtracking Search Trace

The search proceeds depth-first, assigning one doctor at a time and checking constraints as soon as enough variables are bound (forward-checking style constraint propagation combined with backtracking).

Step	Variable Assigned	Constraint Check	Result
1	D1 = Morning	(i) $D1 \neq \text{Night}$ → satisfied	Continue
2	D2 = Morning	(iii) Morning already taken by D1 → conflict	Fail, try next value
3	D2 = Afternoon	(iii) Afternoon free → no clash	Continue

Step	Variable Assigned	Constraint Check	Result
4	D3 = Night	(iv) D3 $\neq$ Morning $\checkmark$ (ii) Afternoon < Night $\checkmark$ (iii) all distinct $\checkmark$	Success — Goal Test Passed
5	D2 = Night (alt.)	(ii) fails: only remaining value Afternoon comes after Night	Fail $\rightarrow$ Backtrack
6	D1 = Afternoon (alt. branch)	D2=Morning, D3=Night satisfies (ii),(iii),(iv)	Alternate valid solution found

2.3 Final Valid Schedule

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

Final Answer: D1 $\rightarrow$ Morning, D2 $\rightarrow$ Afternoon, D3 $\rightarrow$ Night. An alternate valid solution (D1=Afternoon, D2=Morning, D3=Night) also exists, confirming this CSP has more than one solution.

2.4 Program Output

1)

```
Run ass x
Step 1: Trying D1 = Morning (1)
-> Valid partial assignment: {'D1': 1}
Step 2: Trying D2 = Morning (1)
-> [PRUNED / BACKTRACK] Constraint violation at {'D1': 1, 'D2': 1}
Step 3: Trying D2 = Afternoon (2)
-> Valid partial assignment: {'D1': 1, 'D2': 2}
Step 4: Trying D3 = Morning (1)
-> [PRUNED / BACKTRACK] Constraint violation at {'D1': 1, 'D2': 2, 'D3': 1}
Step 5: Trying D3 = Afternoon (2)
-> [PRUNED / BACKTRACK] Constraint violation at {'D1': 1, 'D2': 2, 'D3': 2}
Step 6: Trying D3 = Night (3)
-> Valid partial assignment: {'D1': 1, 'D2': 2, 'D3': 3}

[GOAL REACHED] Valid Assignment Found: {'D1': 1, 'D2': 2, 'D3': 3}

=====
      FINAL VALID SCHEDULE
=====
D1 -> Morning
D2 -> Afternoon
D3 -> Night
```

2)

```
Run ass x
Step 14:
  Popped Node: (2, 3) | g = 5, h = 3, f = 8
  Expanded Neighbor (2, 4): g=6, h=2, f=8

Step 15:
  Popped Node: (1, 4) | g = 5, h = 3, f = 8

Step 16:
  Popped Node: (4, 2) | g = 6, h = 2, f = 8
  Expanded Neighbor (4, 3): g=7, h=1, f=8

Step 17:
  Popped Node: (2, 4) | g = 6, h = 2, f = 8
  Expanded Neighbor (3, 4): g=7, h=1, f=8

Step 18:
  Popped Node: (4, 3) | g = 7, h = 1, f = 8
  Expanded Neighbor (4, 4): g=8, h=0, f=8

Step 19:
  Popped Node: (3, 4) | g = 7, h = 1, f = 8

Step 20:
  Popped Node: (4, 4) | g = 8, h = 0, f = 8

>>> GOAL REACHED! <<<

=====
Optimal Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Total Path Cost: 8
=====
```

3. Question 2 — Robot Grid Navigation (BFS)

3.1 Problem Statement

A robot operates in a 5×5 grid environment and must move from a Start cell (S) to a Goal cell (G), while avoiding obstacles, subject to: (i) moves restricted to Up/Down/Left/Right, (ii) uniform move cost of 1, (iii) obstacles cannot be traversed, and (iv) Manhattan Distance used as the heuristic reference to judge path optimality.

Task: Using BFS, show step-by-step node expansion and priority-queue updates, and determine the optimal path and cost.

3.2 Solution Approach — BFS Trace (Summary)

Starting from S = (0,0), BFS expands nodes level by level in FIFO order, generating neighbours in Right, Down, Left, Up order, skipping obstacles and already-visited cells, until the goal (4,4) is generated.

Step	Node Dequeued	Newly Added to Queue
1	(0,0) S	(0,1),(1,0)
2	(0,1)	(0,2)
4	(0,2)	(0,3),(1,2)
6	(0,3)	(0,4)
9	(0,4)	(1,4)
12	(1,4)	(2,4)
16	(2,4)	(3,4)
18	(3,4)	(4,4) — Goal reached

(Full 18-step trace with all obstacle checks and queue states is provided in the accompanying Solution document.)

3.3 Result

Optimal Path: (0,0) → (0,1) → (0,2) → (0,3) → (0,4) → (1,4) → (2,4) → (3,4) → (4,4)

Path length = 8 moves → Total cost = 8. The Manhattan Distance lower bound was 8, and BFS found a path of cost exactly 8 — confirming optimality.

3.4 Program Output

```
Run ass x
Step 7:
Popped Node: (0, 3) | g = 3, h = 5, f = 8
Expanded Neighbor (1, 3): g=4, h=4, f=8
Expanded Neighbor (0, 4): g=4, h=4, f=8

Step 8:
Popped Node: (4, 0) | g = 4, h = 4, f = 8
Expanded Neighbor (4, 1): g=5, h=3, f=8

Step 9:
Popped Node: (3, 1) | g = 4, h = 4, f = 8
Expanded Neighbor (3, 2): g=5, h=3, f=8

Step 10:
Popped Node: (1, 3) | g = 4, h = 4, f = 8
Expanded Neighbor (2, 3): g=5, h=3, f=8
Expanded Neighbor (1, 4): g=5, h=3, f=8

Step 11:
Popped Node: (0, 4) | g = 4, h = 4, f = 8

Step 12:
Popped Node: (4, 1) | g = 5, h = 3, f = 8
Expanded Neighbor (4, 2): g=6, h=2, f=8

Step 13:
Popped Node: (3, 2) | g = 5, h = 3, f = 8
Expanded Neighbor (2, 2): g=6, h=4, f=10

Run ass
C:\Users\teeka\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\teeka\PycharmProjects\PythonProject\ass.py
=====
A* SEARCH STEP-BY-STEP TRACE
=====
Step 1:
Popped Node: (0, 0) | g = 0, h = 8, f = 8
Expanded Neighbor (1, 0): g=1, h=7, f=8
Expanded Neighbor (0, 1): g=1, h=7, f=8

Step 2:
Popped Node: (1, 0) | g = 1, h = 7, f = 8
Expanded Neighbor (2, 0): g=2, h=6, f=8

Step 3:
Popped Node: (0, 1) | g = 1, h = 7, f = 8
Expanded Neighbor (0, 2): g=2, h=6, f=8

Step 4:
Popped Node: (2, 0) | g = 2, h = 6, f = 8
Expanded Neighbor (3, 0): g=3, h=5, f=8

Step 5:
Popped Node: (0, 2) | g = 2, h = 6, f = 8
Expanded Neighbor (0, 3): g=3, h=5, f=8

Step 6:
Popped Node: (3, 0) | g = 3, h = 5, f = 8
Expanded Neighbor (4, 0): g=4, h=4, f=8
Expanded Neighbor (3, 1): g=4, h=4, f=8
```

4. Question 3 — Autonomous Rescue Robot (Online / UCS Search)

4.1 Problem Statement

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors, operating in a partially observable environment with limited sensing, where movement costs 1 normally and entering risky zones costs an additional 2. The robot must minimize total cost while ensuring safe navigation and updating its knowledge dynamically.

4.2 Effect of Constraints on Decision-Making

- Obstacles remove blocked neighbours from consideration during expansion.
- Limited sensing (adjacent cells only) means the map is discovered incrementally, forcing planning and execution to be interleaved.
- Non-uniform move cost (risky zones cost 3) rules out plain BFS, since it assumes uniform cost.
- Minimizing total cost while staying safe requires expanding the lowest cumulative-cost frontier node first.

4.3 Solution Approach — Online Uniform Cost Search

- Step 1 (Initialize): place S in a priority queue ordered by $g(n)$, with $g(S) = 0$.
- Step 2 (Sense): observe adjacent cells and mark them free, obstacle, or risky-zone in the known map.
- Step 3 (Expand lowest-cost node): pop the frontier node with smallest $g(n)$ and generate its neighbours from the known map only.
- Step 4 (Assign edge costs): 1 for a normal cell, 3 for a risky zone; update $g(n) = g(\text{parent}) + \text{edge cost}$.
- Step 5 (Goal test): terminate when G is expanded, returning the minimum-cost path found.
- Step 6 (Move and repeat): move to the next node, sense again, and re-plan if new information invalidates the current path.

4.4 AI Concepts Demonstrated

- Intelligent Agent: perceives via limited local sensors and acts through movement in a continuous sense–plan–act loop.
- Rationality: always selects the action expected to minimize cumulative cost while avoiding known obstacles.
- Environment type: partially observable, deterministic, sequential, discrete, single-agent.

4.5 Justification

- UCS expands nodes by true cumulative cost $g(n)$, correctly handling variable move costs.
- UCS guarantees the minimum-cost path for non-negative edge costs.
- UCS extends naturally into an online agent that re-searches when new sensing data changes the known map.
- Obstacles are only added to the known map once sensed, so the robot never plans through a blocked cell.

Conclusion: an Online Uniform Cost Search agent is better suited to this scenario than plain BFS (ignores variable move costs) or a fully offline search (assumes complete map knowledge).

4.6 Program Output

3)

```
Run 055
C:\Users\teeka\PycharmProjects\PythonProject\venv\Scripts\python.exe C:\Users\teeka\PycharmProjects\PythonProject\ass.py

=====
ONLINE RESCUE ROBOT NAVIGATION SIMULATION
=====
Step 1: Moved to (1, 0) | Cell: Clear (Cost+1) | Path Cost: 1
Step 2: Moved to (2, 0) | Cell: Clear (Cost+1) | Path Cost: 2
Step 3: Moved to (3, 0) | Cell: Clear (Cost+1) | Path Cost: 3
Step 4: Moved to (4, 0) | Cell: Clear (Cost+1) | Path Cost: 4
Step 5: Moved to (4, 1) | Cell: Clear (Cost+1) | Path Cost: 5
Step 6: Moved to (4, 2) | Cell: Clear (Cost+1) | Path Cost: 6
Step 7: Moved to (4, 3) | Cell: Clear (Cost+1) | Path Cost: 7
Step 8: Moved to (4, 4) | Cell: Clear (Cost+1) | Path Cost: 8

=====
>>> SURVIVOR LOCATED SUCCESSFULLY! <<<
Optimal Path Taken: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Total Path Cost: 8
=====

This view is read-only
Press any key to continue with exit code 0
```

5. Overall Conclusion

Across all three questions, the choice of search strategy was matched to the structure of the underlying problem: Backtracking Search for a discrete constraint-satisfaction assignment problem, BFS for an unweighted shortest-path problem with a known map, and Online Uniform Cost Search for a weighted, partially observable navigation problem. In each case the algorithm's guarantees (constraint consistency, path optimality, and cost-optimality under uncertainty, respectively) were verified both analytically through the step-by-step trace and empirically through the corresponding program output.